Programarea calculatoarelor

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 10 / 15.12.2025

Programa cursului

1. Introducere
- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C

2. Fundamentele limbajului C
- Etapele realizării unui program C
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții
- Funcții de citire/scriere

3. Fișiere text
- Funcții specifice de manipulare

4. Funcții (1)
- Declarare și definire. Apel.
- Metode de trasmitere a paramerilor.
- Pointeri la funcții

5. Tablouri și pointeri
- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere
- Funcții specifice de manipulare

7. Fișiere binare
- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite
- **Definire și utilizare**

9. Funcții (2)
- **Funcții cu număr variabil de argumente**
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

Cuprinsul cursului de azi

1. **Structuri de date complexe și autoreferite**
2. **Funcții cu număr variabil de argumente**

Structuri

- structură = colecție de variabile grupate sub același nume
- **sintaxa:**

```
struct <nume> {
                < tip 1 >     <variabila 1>;
                < tip 2 >     <variabila 2>;
                ------------------------------
                < tip n >     <variabila n>;
                } lista_identificatori_de_tip_struct;
```

- variabilele care fac parte din structură sunt denumite **membri** (elemente sau câmpuri) ai structurii.

Structuri

```
struct <nume> { < tip 1 >    <variabila 1>;
                < tip 2 >    <variabila 2>;
                ------------------------------
                < tip n >    <variabila n>;
               } lista_identificatori;
```

Obs. 1:

- ✓ dacă numele structurii (<nume>) lipsește, structura se numește **anonimă**
- ✓ dacă lista identificatorilor declarați lipsește, se definește doar tipul structură
- ✓ cel puțin una dintre aceste specificații trebuie să existe

Obs. 2: dacă <nume> este prezent => se pot declara noi variabile de tip structură:

struct <nume> <lista noilor identificatori>;

Obs. 3: referirea unui membru al unei variabile de tip structură => **operatorul de selecție punct .** care precizează identificatorul variabilei și al câmpului

Structuri

```
struct student {
        char nume[30];
        char prenume[30];
        float medie_admitere;
} A, B, C;
```

*/*Definește un tip de structură numit **student** și declară ca fiind de acest tip variabilele **A, B, C***/*

```
struct {
        char nume[30];
        char prenume[30];
        float medie_admitere;
} A;
```

*/*Declară o variabilă numită **A** definită de structura care o precede*/*

```
typedef struct {
      char nume[30];
      char prenume[30];
      float medieIntrare;
} student;
student A;
```

*/*Definește un tip de date numit **student** și declară variabila **A** de tip student*/*

Structuri. Inițializare

```
Popescu Ion 9.25
Ionescu Dana 9.25
Popescu  0.00
```

```c
#include <stdio.h>
typedef struct{
    char nume[30];
    char prenume[30];
    float medie_admitere;
} student;

int main()
{
    student A={"Popescu","Ion", 9.25};
    student B={.medie_admitere=9.25,.prenume="Dana",.nume="Ionescu"};
    student C={"Popescu"};

    printf("%s %s %.2f\n", A.nume, A.prenume, A.medie_admitere);
    printf("%s %s %.2f\n", B.nume, B.prenume, B.medie_admitere);
    printf("%s %s %.2f\n", C.nume, C.prenume, C.medie_admitere);

    return 0;
}
```

Ordinea!

Transmiterea structurilor ca parametri

```c
#include <stdio.h>

typedef struct{
    char nume[30];
    char prenume[30];
    float medie_admitere;
}student;

void adauga_un_punct(student x)
{
    x.medie_admitere++;
    if(x.medie_admitere > 10)
        x.medie_admitere = 10;
}
int main()
{
    student A = {"Popescu", "Ion", 9.50};

    adauga_un_punct(A);

    printf("%s %s %.2f\n", A.nume, A.prenume, A.medie_admitere);
    return 0;
}
```

Popescu Ion 9.50

?

Transmiterea structurilor ca parametri

- când o structură este transmisă ca parametru unei funcții se face o copie a zonei de memorie respective
- transmitere prin valoare
- la ieșirea din funcție se distruge copia locală (*x în exemplul anterior*) => modificările efectuate asupra structurii în funcție **nu** vor afecta și structura originală

- Soluția: **pointeri la structuri**
  - ✓folosim operatorul → pentru a accesa câmpurile

Transmiterea structurilor ca parametri. Pointeri la structuri

```c
#include <stdio.h>

typedef struct{
    char nume[30];
    char prenume[30];
    float medie_admitere;
}student;

void adauga_un_punct(student *x)
{
    x->medie_admitere++;
    if(x->medie_admitere > 10)
        x->medie_admitere = 10;
}
int main()
{
    student A = {"Popescu", "Ion", 9.50};

    adauga_un_punct(&A);

    printf("%s %s %.2f\n", A.nume, A.prenume, A.medie_admitere);
    return 0;
}
```

Popescu Ion 10.00

Pointeri la structuri

- folosim operatorul → pentru a accesa câmpurile
- respectă aceleași reguli ca și ceilalți pointeri
- trebuie să fie inițializați
- trebuie să se facă conversii explicite când este cazul
- pointeri la structuri vs. structuri care conțin pointeri

Structuri imbricate și tablouri de structuri

- **structură imbricată** (nested) = o structură care conține ca membru o altă structură
- **tablou de structuri** = tablou cu elemente de tip struct

```
struct student {
    char nume[30];
    char prenume[30];
    float medie_admitere;
    struct adresa; /*Structura adresa trebuie să fie definită în prealabil*/
} ;
struct  student grupa[30]; /*tablou de structuri*/
```

Structuri imbricate și tablouri de structuri

Enunț:

Fișierul text *triunghi.txt* conține pe prima linie un număr natural n (n > 0), apoi n linii. Fiecare linie conține coordonatele reale (abscisa și ordonata) a 3 puncte date sub forma:

abscisa1 ordonata1 abscisa2 ordonata2 abscisa3 ordonata3

Să se afișeze aria celui mai mare triunghi, dacă acesta există, sau mesajul “nu există” dacă nici un triplet de puncte de pe o linie nu poate defini un triunghi.

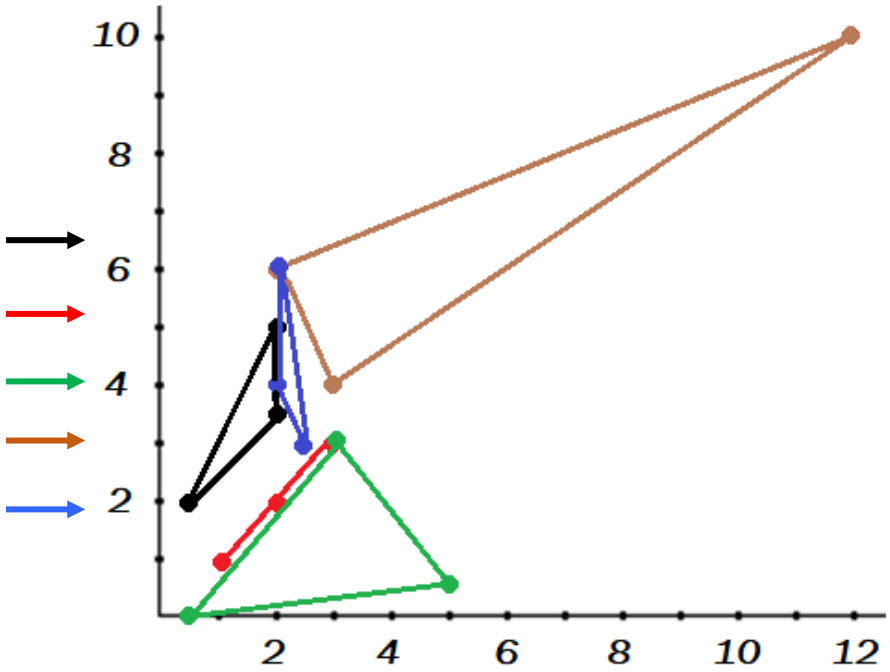
Structuri imbricate și tablouri de structuri

Fișierul text *triunghi.txt* conține pe prima linie un număr natural n (n > 0), apoi n linii. Fiecare linie conține coordonatele reale (abscisa și ordonata) a 3 puncte date sub forma:

abscisa1 ordonata1 abscisa2 ordonata2 abscisa3 ordonata3

main.c | triunghiuri.txt

```
1  5
2  0.5 2 2 3.5 2 5
3  1 1 2 2 3 3
4  0.5 0 3 3 5 0.5
5  12 10 3 4 2 6
6  2.5 3 2 4 2 6
```



Structuri imbricate și tablouri de structuri

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    float ox; // abscisa
    float oy; // ordonata
} punct;

typedef struct {
    punct A, B, C;
    float AB, AC, BC;
    float perimetru;
    float arie;
} triunghi;
```

```c
int citeste_triunghi(char *, triunghi **);
int afisare_triunghiuri(triunghi *, int);
void afis_arie_maxima(triunghi *pT, int n);

int main()
{
    char nume_fisier[] = "triunghiuri.txt";
    triunghi *T = NULL;
    int n = citeste_triunghi(nume_fisier, &T);
    afisare_triunghiuri(T, n);
    afis_arie_maxima(T, n);

    return 0;
}
```

```c
int citeste_triunghi(char *nume, triunghi **pT)
{
    FILE *f = fopen(nume, "r");
    if (f == NULL) { printf("Eroare la deschidere fisier."); exit(0);}

    int n; fscanf(f, "%d", &n);
    triunghi *p = (triunghi*) malloc (n * sizeof(triunghi));
    if (p == NULL) { printf("Eroare la alocare."); exit(0);}

    punct A, B, C;
    for(int i = 0; i < n; i++)  {
        fscanf(f, "%f %f %f %f %f %f", &A.ox, &A.oy, &B.ox, &B.oy, &C.ox, &C.oy);

        float AB = sqrt(pow(A.ox - B.ox, 2) + pow(A.oy - B.oy, 2));
        float AC = sqrt(pow(A.ox - C.ox, 2) + pow(A.oy - C.oy, 2));
        float BC = sqrt(pow(B.ox - C.ox, 2) + pow(B.oy - C.oy, 2));

        p[i].A = A, p[i].B = B, p[i].C = C, p[i].AB = AB, p[i].AC = AC, p[i].BC = BC;

        if(AB + BC == AC || AB + AC == BC || AC + BC == AB)
            p[i].arie = 0;
        else
        {
           p[i]. perimetru = AB + BC + AC;
           float sp = p[i]. perimetru / 2;
           p[i].arie = sqrt(sp * (sp - p[i].AB) * (sp - p[i].AC) * (sp - p[i].BC));
        }
    }
    *pT = p;
    fclose(f);
    return n;
}
```

```c
int afisare_triunghiuri(triunghi *pT, int n)
{
    for(int i = 0; i < n; i++)
    {
        printf("\nSet de date %d: \n", i + 1);

        printf("punctul A(%.2f, %.2f)\n", pT[i].A.ox, pT[i].A.oy);
        printf("punctul B(%.2f, %.2f)\n", pT[i].B.ox, pT[i].B.oy);
        printf("punctul C(%.2f, %.2f)\n", pT[i].C.ox, pT[i].C.oy);

        printf("|AB| = %.2f\n",pT[i].AB);
        printf("|AC| = %.2f\n",pT[i].AC);
        printf("|BC| = %.2f\n",pT[i].BC);

        if(pT[i].arie > 0)
            printf("aria triunghiului = %.2f\n", pT[i].arie);
        else
            printf("Punctele sunt coliniare => nu formeaza triunghi.\n");
    }
}
```

```
 86  void afis_arie_maxima(triunghi *pT, int n)
 87  {
 88      float arie_max = 0;
 89      int set = 0;
 90
 91      for(int i = 0; i < n; i++)
 92          if(pT[i].arie > arie_max)
 93          {
 94              arie_max = pT[i].arie;
 95              set = i + 1;
 96          }
 97
 98      if(set)
 99          printf("\naria maxima = %.2f/ set de date %d\n", arie_max, set);
100      else
101          printf("Niciun set de date valid!\n");
102  }
```

main.c | triunghiuri.txt

```
5
0.5 2 2 3.5 2 5
1 1 2 2 3 3
0.5 0 3 3 5 0.5
12 10 3 4 2 6
2.5 3 2 4 2 6
```

```
Set de date 1:
punctul A(0.50, 2.00)
punctul B(2.00, 3.50)
punctul C(2.00, 5.00)
|AB| = 2.12
|AC| = 3.35
|BC| = 1.50
aria triunghiului = 1.12

Set de date 2:
punctul A(1.00, 1.00)
punctul B(2.00, 2.00)
punctul C(3.00, 3.00)
|AB| = 1.41
|AC| = 2.83
|BC| = 1.41
Punctele sunt coliniare => nu formeaza triunghi.
```

```
Set de date 3:
punctul A(0.50, 0.00)
punctul B(3.00, 3.00)
punctul C(5.00, 0.50)
|AB| = 3.91
|AC| = 4.53
|BC| = 3.20
aria triunghiului = 6.13

Set de date 4:
punctul A(12.00, 10.00)
punctul B(3.00, 4.00)
punctul C(2.00, 6.00)
|AB| = 10.82
|AC| = 10.77
|BC| = 2.24
aria triunghiului = 12.00

Set de date 5:
punctul A(2.50, 3.00)
punctul B(2.00, 4.00)
punctul C(2.00, 6.00)
|AB| = 1.12
|AC| = 3.04
|BC| = 2.00
aria triunghiului = 0.50

aria maxima = 12.00/ set de date 4
```

main.c | triunghiuri.txt

```
2
0.5 0.5 2 2 5 5
1 1 2 2 3 3
```

```
Set de date 1:
punctul A(0.50, 0.50)
punctul B(2.00, 2.00)
punctul C(5.00, 5.00)
|AB| = 2.12
|AC| = 6.36
|BC| = 4.24
Punctele sunt coliniare => nu formeaza triunghi.

Set de date 2:
punctul A(1.00, 1.00)
punctul B(2.00, 2.00)
punctul C(3.00, 3.00)
|AB| = 1.41
|AC| = 2.83
|BC| = 1.41
Punctele sunt coliniare => nu formeaza triunghi.
Niciun set de date valid!
```

Sortarea unui tablou de structuri

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    float ox; // abscisa
    float oy; // ordonata
} punct;

typedef struct {
    punct A, B, C;
    float AB, AC, BC;
    float perimetru;
    float arie;
} triunghi;
```

```c
int main()
{
    char nume_fisier[] = "triunghiuri.txt";
    triunghi *T = NULL;
    int n = citeste_triunghi(nume_fisier, &T);
    afisare_triunghiuri(T, n);
    //sorteaza_triunghiuri(T, n);
    qsort(T, n, sizeof(triunghi), cmp);
    afisare_triunghiuri(T, n);

    free(T);
    return 0;
}
```

```c
void sorteaza_triunghiuri(triunghi *pT, int n)
{
    int i, j; triunghi aux;
    for(i = 0; i < n - 1; i++)
        for(j = i +1; j < n; j++)
            if(pT[i].arie > pT[j]. arie)
            {
                aux = pT[i];
                pT[i] = pT[j];
                pT[j] = aux;
            }
}

int cmp(const void* a, const void* b)
{
    triunghi *a1 = (triunghi *)a;
    triunghi *b1 = (triunghi *)b;

    if (a1->arie < b1->arie)
        return -1;      // ordine crescătoare
    else if (a1->arie > b1->arie)
        return 1;
    else
        return 0;
}
```

Structuri autoreferite

- **structuri autoreferite** = structuri care conțin o declarație recursivă pentru anumiți membri de tip pointer

```
struct  T{
            char ch;
            int i;
            struct T *t;
            };
```
declarație validă

```
struct  T{
            char ch;
            struct S *t;
            };
```

```
struct S{
            int i;
            struct T *q;
            };
```

declarație validă: structurile S și T se invocă reciproc

- este ilegal ca o structură să conțină o instanțiere a sa

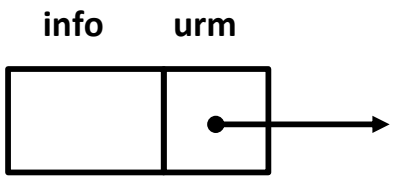
```
struct  T{
            char ch;
            int i;
            struct T t;
            };
```
Declarație invalidă

Structuri autoreferite

- aplicații pentru structuri de date în alocare dinamică:

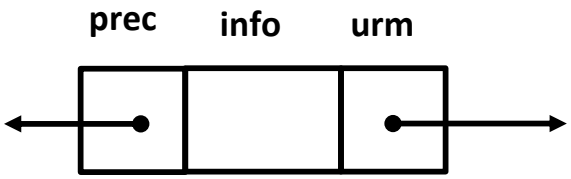
listă simplu înlănțuită:

```
struct  nod{
        int info;
        struct nod *urm;
};
```



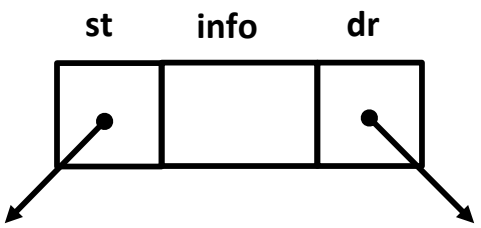
listă dublu înlănțuită:

```
struct  nod{
        int info;
        struct nod *urm;
        struct nod *prec;
};
```



arbore binar:

```
struct  nod{
        int info;
        struct nod *fiuSt;
        struct nod *fiuDr;
};
```



- fiecare nod conţine:
  - ✓ un câmp/mai multe câmpuri cu informația nodului - info
  - ✓ un pointer/mai mulți pointeri către nodurile vecine: următor/succesor, precedent/predecesor, fiuStang, fiuDrept

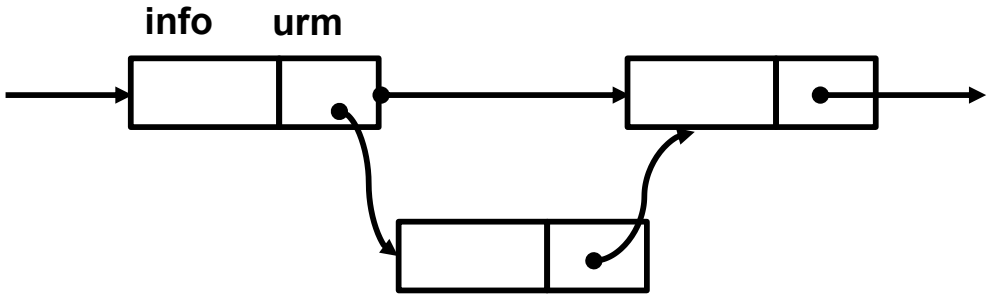
Structuri autoreferite

- aplicații pentru structuri de date în alocare dinamică

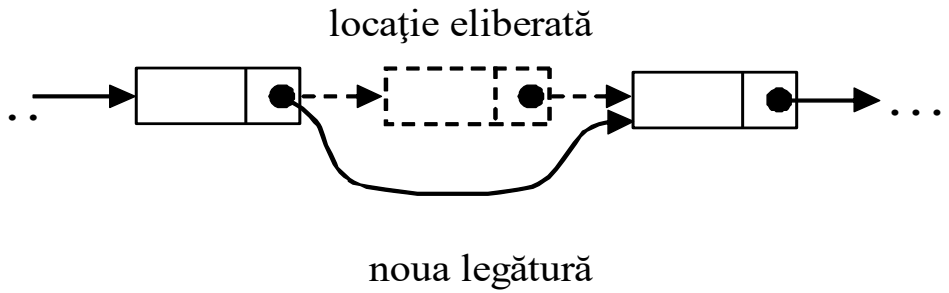
- **liste, stive, cozi, arbori**
  - ✓ avantaj față de implementarea statică:
    - operațiile de adăugare sau ștergere sunt foarte rapide
  - ✓ dezavantaj față de implementarea statică :
    - accesul la un nod se face prin parcurgerea nodurilor precedente
    - adresa nodurilor vecine ocupă memorie suplimentară

Structuri autoreferite

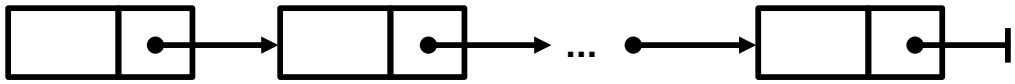
- aplicații pentru structuri de date în alocare dinamică
- **Operații specifice:**
  - ✓adăugare



  - ✓ștergere



  - ✓traversare
  - ✓căutare



Exemple: adăugare, ștergere, traversare-căutare în listă simplu înlănțuită

Structuri autoreferite

Exemplu: aplicație pentru structuri de date în alocare dinamică:

- operații cu **vectori rari**: suma și produsul scalar a doi vectori rari
  - ✓ multe dintre elementele vectorului egale cu 0
  - ✓ reprezentare eficientă: liste simplu înlănțuite alocate dinamic
  - ✓ fiecare nod din listă reține:
    - valoarea
    - poziția din vector pe care se găsește elementul nenul
    - adresa nodului următor

```
struct  nod{
            float info;
            int poz;
            struct nod *urm;
            };
/*declarație a structurii  nod folosită
la reprezentarea listei*/
```

- citesc vectorii din două fișiere text ce specifică valoarea și poziția elementelor nenule din fiecare vector

Structuri autoreferite

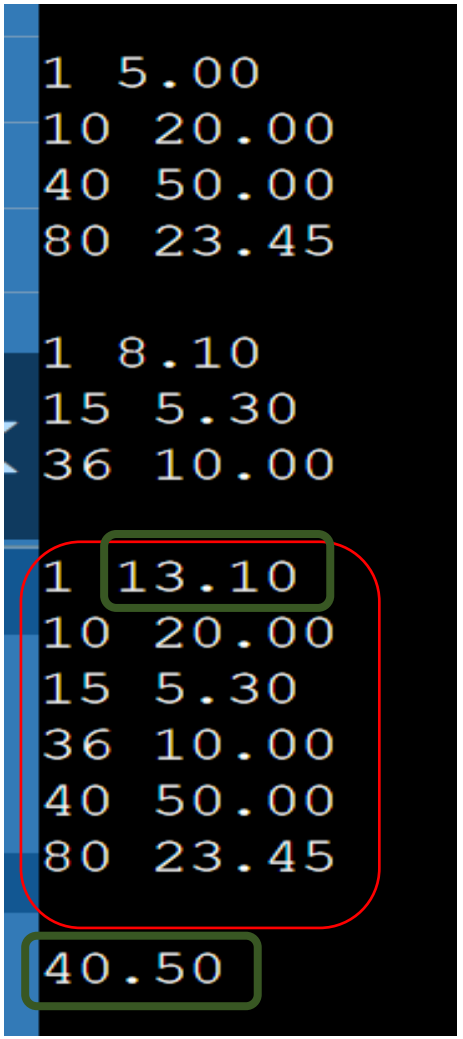
Operații cu vectori rari: suma și produsul scalar a doi vectori rari

vector1.txt
1 5
10 20
40 50
80 23.45

vector2.txt
1 8.1
15 5.3
36 10

Valoarea

Poziția



Structuri autoreferite

Operații cu vectori rari: suma și produsul scalar a doi vectori rari

main.c | vector1.txt | vector2.txt

```
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  typedef struct{
8      float info;
9      int poz;
10     struct nod *urm;
11 }nod;
12
13 void adaugare(nod**, nod**, float, int);
14 void construire_lista(nod**, nod**,char*);
15 void afisare_lista(nod*);
16 void suma(nod*, nod*, nod**, nod**);
17 float produs_scalar(nod*,nod*);
18
19 int main()
```

```
19  int main()
20  {
21      char* nume_fisier1="vector1.txt";
22      char* nume_fisier2="vector2.txt";
23      /*atentie: pt un spatiu in plus la finalul fisierului
24      dubleaza ultimul nod*/
25
26      nod* prim1=NULL; nod* ultim1=NULL;
27      construire_lista(&prim1,&ultim1,nume_fisier1);
28      afisare_lista(prim1);
29
30      nod* prim2=NULL; nod* ultim2=NULL;
31      construire_lista(&prim2,&ultim2,nume_fisier2);
32      afisare_lista(prim2);
33
34      nod* prim3=NULL; nod* ultim3=NULL;
35      suma(prim1,prim2, &prim3, &ultim3);
36      afisare_lista(prim3);
37
38      printf("\n%.2f",produs_scalar(prim1,prim2));
39
40      return 0;
41  }
```

```
43  void construire_lista(nod** p, nod** u,char* nume_fisier)
44  {
45      FILE* f=fopen(nume_fisier, "r");
46      if (f==NULL)
47          {printf("Eroare la deschidere fisier\n");exit(0);}
48      int poz; float val;
49      while(!feof(f))
50          {
51              fscanf(f,"%d %f",&poz, &val);
52              adaugare(p, u, val, poz);
53          }
54  }
55
56  void afisare_lista(nod* p)
57  {
58      printf("\n");
59      while(p)
60      {
61          printf("%d %.2f\n", p->poz, p->info);
62          p=p->urm;
63      }
64  }
```

```
66  void adaugare(nod** p, nod** u, float val, int poz)
67  {
68      if(*p==NULL)
69      {
70          *p=(nod*)malloc(sizeof(nod));
71          (*p)->info=val;
72          (*p)->urm=NULL;
73          (*p)->poz=poz;
74          *u=*p;
75      }
76      else
77      {
78          nod* c=(nod*)malloc(sizeof(nod));
79          c->info=val;
80          c->urm=NULL;
81          c->poz=poz;
82          (*u)->urm=c;
83          *u=c;
84      }
85  }
86
```

```
87  void suma(nod* prim1, nod* prim2, nod** prim3, nod** ultim3)
88  {
89      nod *p1,*p2; p1=prim1; p2=prim2;
90      while(p1!=NULL &&p2!=NULL)
91      {
92          if(p1->poz < p2->poz)
93          {   adaugare(prim3,ultim3,p1->info,p1->poz); p1=p1->urm;
94          }
95          else
96              if(p1->poz > p2->poz)
97              {   adaugare(prim3,ultim3,p2->info,p2->poz); p2=p2->urm;
98              }
99          else    //if(p1->poz == p2->poz)
100             {   adaugare(prim3,ultim3,p1->info + p2->info,p2->poz);
101                 p1=p1->urm;p2=p2->urm;
102             }
103     }
104     while(p1!=NULL)
105     {   adaugare(prim3,ultim3,p1->info,p1->poz); p1=p1->urm;
106     }
107     while(p2!=NULL)
108     {   adaugare(prim3,ultim3,p2->info,p2->poz); p2=p2->urm;
109     }
110 }
```

```
112 float produs_scalar(nod* prim1,nod* prim2)
113 {
114     float r=0;
115     nod *p1,*p2; p1=prim1; p2=prim2;
116     while(p1!=NULL &&p2!=NULL)
117     {
118         if(p1->poz == p2->poz)
119             {
120                 r+=(p1->info)*(p2->info);
121                 p1=p1->urm;
122                 p2=p2->urm;
123             }
124         else
125             if(p1->poz < p2->poz)
126                 p1=p1->urm;
127             else
128                 if(p1->poz > p2->poz)
129                     p2=p2->urm;
130     }
131     return r;
132 }
```

Cuprinsul cursului de azi

1. Structuri de date complexe și autoreferite
2. **Funcții cu număr variabil de argumente**

Funcții cu număr variabil de argumente

- funcții cu număr variabil de argumente utilizate de voi până acum:
  - ✓ **printf, fprintf**
  - ✓ **scanf, fscanf**

Funcții cu număr variabil de argumente

- probleme:
  - nu se cunoaște numărul parametrilor funcției
  - nu se cunoaște tipul parametrilor

- header-ul **stdarg.h** cuprinde:
  - ✓ definiția unui **tip de date specializat (va_list)** dedicat manipulării listelor cu număr variabil de parametri
  - ✓ **macro-uri (va_start, va_arg, va_end)** care realizează operații pe acest tip de date

macrou = fragment de cod căruia i se asociază un nume, la preprocesare se înlocuiește numele cu fragmentul de cod

Directiva #define. Exemplu (Curs 5)

```
1  #include <stdio.h>
2
3  #define ALPHA 30
4  #define BETA ALPHA+10
5  #define GAMMA (ALPHA+10)
6
7  #define MIN(a, b) ((a) < (b))? (a) : (b)
8  #define ABS1(x) (x < 0)? -x : x
9  #define ABS2(x) (((x) < 0)? -(x) : (x))
10 #define INTER(tip, a, b)\
11     {tip c; c = a; a = b; b = c;}
12
```

constante simbolice

macro-uri

```
13 int main()
14 {
15     int x = 2*BETA;
16     int y = 2*GAMMA;
17     printf("%d, %d\n", x, y); // 70 80
18     int m = MIN(x, y);
19     printf("%d\n",m);    // 70
20     int a = ABS1(x - y);
21     int b = ABS2(x - y);
22     printf("%d, %d\n\n", a, b);// -150 10
23     INTER(int, a, b);
24     printf("%d, %d\n", a, b);  //10 -150
25     INTER(int, a, b);
26     printf("%d, %d\n", a, b);  // -150 10
```

Funcții cu număr variabil de argumente

Sintaxa:

tip_rezultat nume_functie(tip_argument nume_argument, …)

unde (***cel puțin***) **primul argument este întotdeauna fix, vizibil**,

restul argumentelor fiind declarate prin *cele trei puncte*, (...) – *mecanismul elipsă*

***Exemplu*:** funcție ce calculează suma a n numere întregi:

int suma(int n,…);

suma(4, 1, 2, 1, 1) -> 5; suma(5, 1, 2, 1, 1, 3) ->8

Funcții cu număr variabil de argumente

În stdarg.h:

- **va_list**: tip de date dedicat manipulării listelor cu număr variabil de parametri (de obicei e unsigned char*)

✓ Macrouri:

- **va_start(va_list lp, numeArgument)** : extrage în lista **lp** parametrii funcției care urmează după ultimul parametru fix specificat de **numeArgument**;
- **va_arg(va_list lp, tip_de_date):** extrage la fiecare apel câte o valoare din lista lp – valoarea se consideră de tipul tip_de_date indicat ca parametru (poate fi int sau double);
- **va_end(va_list lp): obligatoriu** când se încheie operațiile pe lista **lp**

Funcții cu număr variabil de argumente

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
int suma(int n, ...)
{
    int i,s=0;
    va_list listaParametri;
    va_start(listaParametri, n);
    for(i=0;i<n;i++)
        s=s + va_arg(listaParametri, int);
    va_end(listaParametri);
    return s;
}

int main() {
    int a;
    a=suma(4,1,2,1,1);
    printf("a=%d\n", a);
    a=suma(5,1,2,1,1,3);
    printf("a=%d\n", a);
    return 0;
}
```

Exemplul 1: funcție ce calculează suma a n numere întregi

```
a=5
a=8
```

Funcții cu număr variabil de argumente

```
void f(int x, …){
 va_list lp; //declara lista de parametri lp
 va_start(lp , x); /*initializeaza lista de parametri,
                     trebuie sa stiu unde incepe, dupa x */
 for( ; ; ){
 tip_de_date t = va_arg(lp, tip_de_date); //extrage parametrul curent
      … }
va_end(lp); }
```

- apelăm funcția f dintr-o altă funcție g;
- f trebuie să știe ce parametri primește;
- folosim va_start care apelează ultimul parametru formal cunoscut (x) transmis și reținut în stivă;
- transmiterea parametrilor se face de la stânga la dreapta într-o stivă.

Funcții cu număr variabil de argumente

```
void f(int x, …){
        va_list lp;
        va_start(lp , x);
         for( ; ; ){
                tip_de_date t = va_arg(lp, tip_de_date);
                … }
         va_end(lp); }
```

- va_start găsește adresa lui x din stivă (e macrou și nu funcție, are acces la stiva bună!) și apoi din stivă ia fiecare argument transmis cu ajutorul lui va_arg;
- va_arg trebuie să știe ce dimensiune în octeți are parametrul pe care trebuie să îl extragă din stivă;
- va_end este obligatoriu, altfel rezultatul e “undefined”;
- funcția f trebuie să știe unde se oprește cu citirea parametrilor

Funcții cu număr variabil de argumente

- posibilă definire a macrou-rilor va_list, va_start, va_arg:

```
typedef unsigned char * va_list;
#define  va_start(lp,param) (lp = (((va_list)&param) + sizeof(param)))
#define va_arg(lp,type) (*(type *)((lp += sizeof(type)) – sizeof(type)))
```

- va_list e un pointer la char (adresă de variabilă stocată pe un octet);
- va_start inițializează lista de argumente ca un pointer ce reține adresa imediată după ultimul parametru formal transmis în stivă. De la această adresa încep parametrii în număr variabil;
- va_arg realizează două lucruri:
  - ✓ updatează lista de argumente la următorul argument mutând pointerul (aritmetica pointerilor);
  - ✓ returnează valoarea argumentului actual (se întoarce);

Funcții cu număr variabil de argumente

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
int suma(int n, ...)
{
    int i,s=0;
    va_list listaParametri;
    va_start(listaParametri, n);
    for(i=0;i<n;i++)
        s=s + va_arg(listaParametri, int);
    va_end(listaParametri);
    return s;
}

int main() {
    int a;
    a=suma(4,1,2,1,1);
    printf("a=%d\n", a);
    a=suma(5,1,2,1,1,3);
    printf("a=%d\n", a);
    return 0;
}
```

Exemplul 1: funcție ce calculează suma a n numere întregi

```
a=5
a=8
```

Funcții cu număr variabil de argumente

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

int suma(int n, ...)
{
    int i,s=0;
    va_list listaParametri;
    va_start(listaParametri, n);
    for(i=0;i<n;i++)
        s=s + va_arg(listaParametri, int);
    va_end(listaParametri);
    return s;
}

int main()
{
    int a;
    a=suma(4,1,2,1,1);     printf("a=%d\n", a);
    a=suma(5,1,2,1,1,3);   printf("a=%d\n", a);
    a=suma(2,1,2,1,1,3);   printf("a=%d\n", a);
    a=suma(10,1,2,1,1,3); printf("a=%d\n", a);
    return 0;
}
```

Număr insuficient de argumente!

Exemplul 1: funcție ce calculează suma a n numere întregi

```
a=5
a=8
a=3
a=-1216864871
```

Funcții cu număr variabil de argumente

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdarg.h>
4
5  int suma(int x, ...)
6  {
7      int t,s;
8      va_list listaParametri;
9      va_start(listaParametri, x);
10     s=x;
11     do
12     {
13         t = va_arg(listaParametri, int);
14         s = s + t;
15     }while (t!=0);
16     va_end(listaParametri);
17     return s;
18 }
19
20 int main()
21 {
22     int a;
23     a=suma(4,1,2,1,1,0);
24     printf("a=%d\n", a);
25     return 0;
26 }
```

Exemplul 2: functie ce calculeaza suma unui șir de numere întregi ce se temină cu 0

```
a=9
```

Funcții cu număr variabil de argumente

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>

int maxim(int n, ...)
{
    int max, i, aux;
    va_list lp;
    va_start(lp, n);
    max = va_arg(lp, int);
    for(i=2;i<=n;i++)
    {
        aux = va_arg(lp, int);
        if(max < aux)
            max = aux;
    }
    va_end(lp);
    return max;
}

int main()
{
    int a = maxim(7,1,2,10,5,7,4,3);
    printf("a=%d\n", a);
    return 0;
}
```

Exemplul 3: funcție ce calculează maximum a n numere întregi

```
a=10
```

Funcții cu număr variabil de argumente

Exemplul 4: funcție pentru concatenarea unui număr varibil de șiruri de caractere într-un singur șir alocat dinamic. *Marcăm sfârșitul șirurilor printr-un șir vid.*

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

char *concatSiruri(const char *, ...);

int main()
{
    char *prim = "Functiile cu numar variabil ";
    char *secund = "de argumente sunt ";
    char *f = concatSiruri(prim, secund, "foarte simple!", (char *)'\0');
    printf("  %s\n", f);

    return 0;
}
```

```
Functiile cu numar variabil de argumente sunt foarte simple!
```

Exemplul 4: funcție pentru concatenarea unui număr varibil de șiruri de caractere într-un singur șir alocat dinamic. *Marcăm sfârșitul șirurilor printr-un șir vid.*

```
18  char *concatSiruri(const char *primulSir, ...)
19  {
20      va_list listaParametri;
21      char *p, *q;
22
23      if(primulSir == NULL) return NULL;
24
25      int lungimeSir = strlen(primulSir);
26      va_start(listaParametri, primulSir);
27      while((p = va_arg(listaParametri, char *)) != NULL)
28          lungimeSir += strlen(p);
29      va_end(listaParametri);
30      q = (char *)malloc(lungimeSir + 1);
31      if (q == NULL) return NULL;
32
33      strcpy(q, primulSir);
34      va_start(listaParametri, primulSir);
35      while((p = va_arg(listaParametri, char *)) != NULL)
36          strcat(q, p);
37      va_end(listaParametri);
38      return q;
39  }
```

Cursul 10

1. Structuri de date complexe și autoreferite
2. Funcții cu număr variabil de argumente

Cursul 11

1. Declarații complexe
2. Preluarea argumentelor funcției main din linia de comandă
3. Programare generică

Chestiuni organizatorice

- *Data propusă* pentru examenul scris:

  21/ 30 ianuarie 2025, ora 10.00-12.00

- Regulament de evaluare și notare:

$$Nota = round(min\ (\mathbf{10}, \underbrace{\textit{Examen}}_{\textbf{3-6p}} + \underbrace{\textit{Laborator}}_{\textbf{2-4p}} + \underbrace{\textit{Seminar}}_{\textbf{0-1p}}), 0)$$

*Vezi și Cursul 1!

Chestiuni organizatorice

- Regulament de evaluare și notare

Nota = *round(min* (**10,** ***Examen*** + ***Laborator*** + ***Seminar***), 0)

3-6p **2-4p** **0-1p**

Exemplu:

Nota la testul scris (examen) = 7.35

Nota laborator = 9.25

Punctaj seminar = 3.5

Nota finală = round(min (10, 7.35*0.6 + 9.25*0.4 + 3.5*0.1), 0) =

= round(min(10, 8.46), 0) = round(8.46, 0) = **8**

Exemplu:

Nota la testul scris = **4.65 (< 5.00)**

Nota laborator = 9.00

Punctaj seminar = 6.5

Nota finală = 4

*Vezi și Cursul 1!

Chestiuni organizatorice

- Laborator=: maximum 4puncte
- Seminar =: maximum 1punct (nr_prezențe*0.5 +activitate)
- Examen =: notă de la 1 la 10 (1 punct din oficiu); Nota 10 = 6puncte

- NU intrați în examen (/restanță) dacă:
  - Nu aveți cel puțin jumătate din punctajul de laborator (2p din 4p)

- Aveți restanță dacă:
  - Nu intrați în examen
  - Nu vă prezentați la examen
  - Nu luați cel puțin nota 5 (fără rotunjire) la testul scris (examen)
- Restanța: se păstrează punctele obținute pe parcursul semestrului la seminar și laborator.

*Vezi și Cursul 1!